7

BREEDING GIANT RATS WITH GENETIC ALGORITHMS

Genetic algorithms are general-purpose optimization programs designed to solve complex problems. Invented in the 1970s, they belong to the class of *evolutionary algorithms*, so named because they mimic the Darwinian process of natural selection. They are especially useful when little is known about a problem, when you're dealing with a nonlinear problem, or when searching for brute-force-type solutions in a large search space. Best of all, they are easy algorithms to grasp and implement.

In this chapter, you'll use genetic algorithms to breed a race of super-rats that can terrorize the world. After that, you'll switch sides and help James Bond crack a high-tech safe in a matter of seconds. These two projects should give you a good appreciation for the mechanics and power of genetic algorithms.

Finding the Best of All Possible Solutions

Genetic algorithms *optimize*, which means that they select the best solution (with regard to some criteria) from a set of available alternatives. For example, if you're looking for the fastest route to drive from New York to Los Angeles, a genetic algorithm will never suggest you fly. It can choose

only from within an allowed set of conditions that *you* provide. As optimizers, these algorithms are faster than traditional methods and can avoid premature convergence to a suboptimal answer. In other words, they efficiently search the solution space yet do so thoroughly enough to avoid picking a good answer when a better one is available.

Unlike *exhaustive* search engines, which use pure brute force, genetic algorithms don't try every possible solution. Instead, they continuously grade solutions and then use them to make "informed guesses" going forward. A simple example is the "warmer-colder" game, where you search for a hidden item as someone tells you whether you are getting warmer or colder based on your proximity or search direction. Genetic algorithms use a fitness function, analogous to natural selection, to discard "colder" solutions and build on the "warmer" ones. The basic process is as follows:

1. Randomly generate a population of solutions.
2. Measure the fitness of each solution.
3. Select the best (warmest) solutions and discard the rest.
4. Cross over (recombine) elements in the best solutions to make new solutions.
5. Mutate a small number of elements in the solutions by changing their value.
6. Return to step 2 and repeat.

The select–cross over–mutate loop continues until it reaches a *stop condition*, like finding a known answer, finding a "good enough" answer (based on a minimum threshold), completing a set number of iterations, or reaching a time deadline. Because these steps closely resemble the process of evolution, complete with survival of the fittest, the terminology used with genetic algorithms is often more biological than computational.

Project #13: Breeding an Army of Super-Rats

Here's your chance to be a mad scientist with a secret lab full of boiling beakers, bubbling test tubes, and machines that go "BZZZTTT." So pull on some black rubber gloves and get busy turning nimble trash-eating scavengers into massive man-eating monsters.

THE OBJECTIVE

Use a genetic algorithm to simulate breeding rats to an average weight of 110 pounds.

Strategy

Your dream is to breed a race of rats the size of bullmastiffs (we've already established that you're mad). You'll start with *Rattus norvegicus*, the brown rat, then add some artificial sweeteners, some atomic radiation from the 1950s, a lot of patience, and a pinch of Python, but no genetic engineering—you're old-school, baby! The rats will grow from less than a pound to a terrifying 110 pounds, about the size of a female bullmastiff (see **Figure 7-1**).

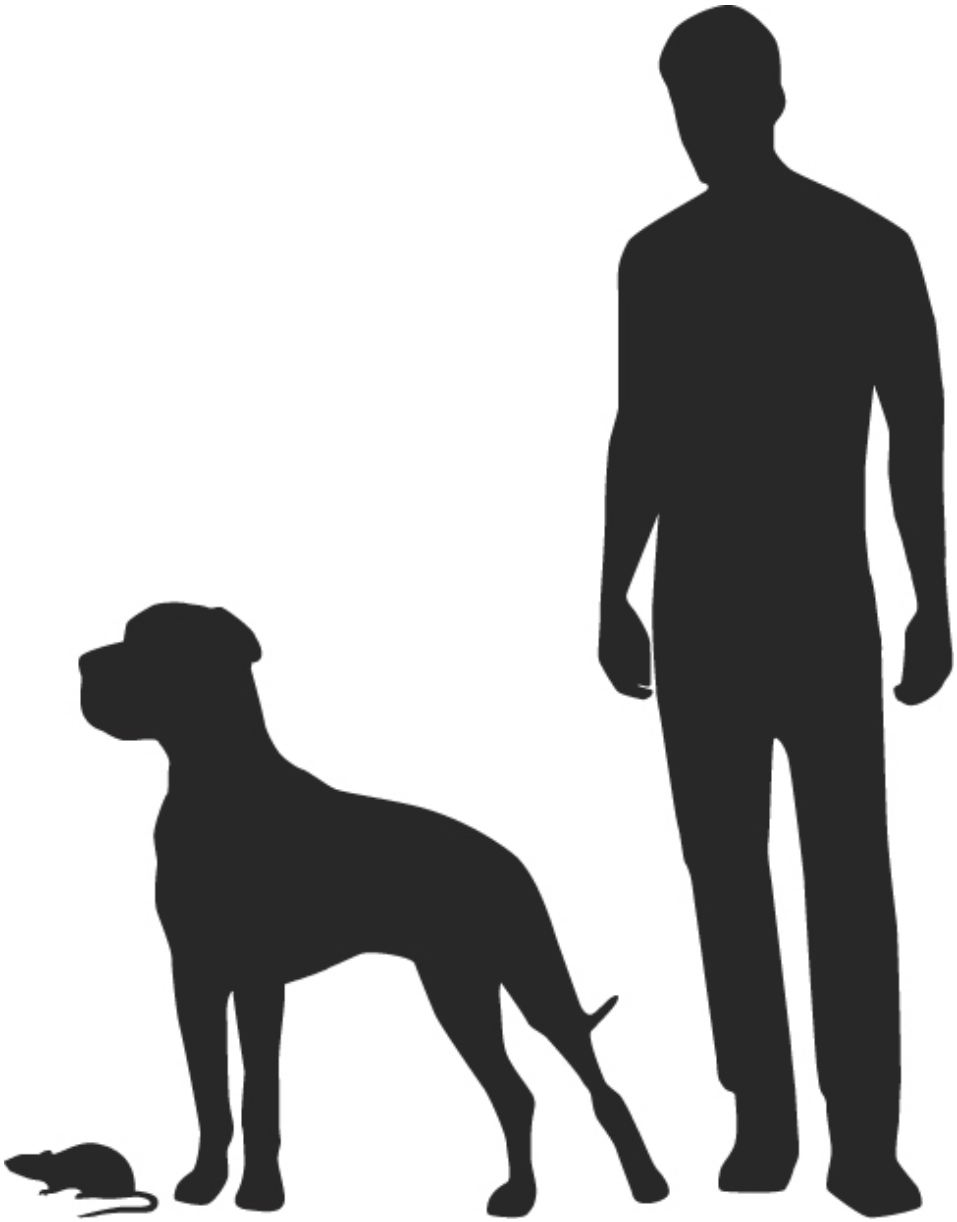

Figure 7-1: Size comparison of a brown rat, a female bullmastiff, and a human

Before you embark on such a huge undertaking, it's prudent to simulate the results in Python. And you've drawn up something better than a plan —you've drawn some graphical pseudocode (see **Figure 7-2**).

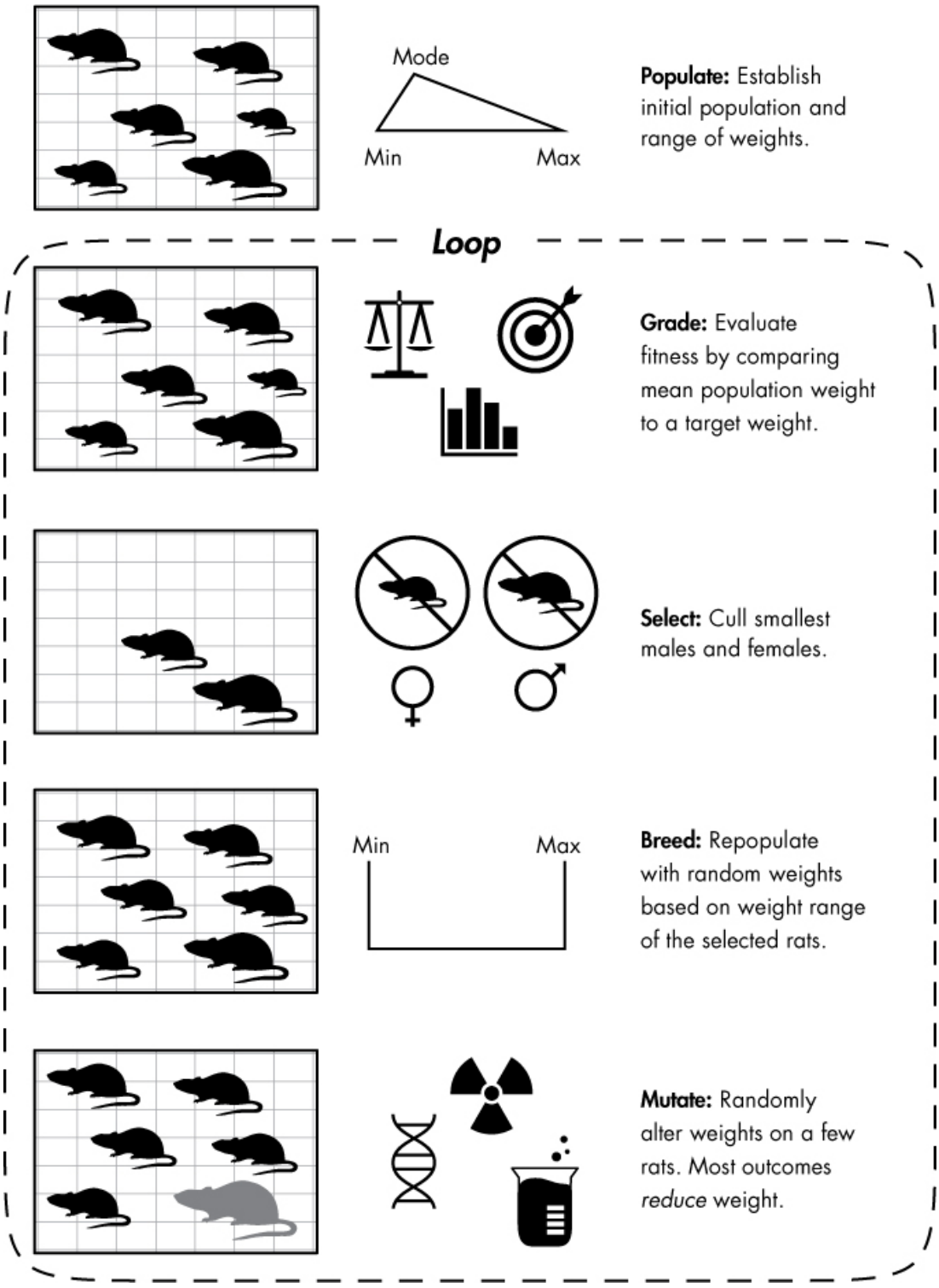

Figure 7-2: Genetic algorithm approach to breeding super-rats

The process shown in **Figure 7-2** outlines how a genetic algorithm works. Your goal is to produce a population of rats with an average weight of 110 pounds from an initial population weighing much less than that. Going forward, each population (or *generation*) of rats represents a candidate solution to the problem. Like any animal breeder, you cull undesirable males and females, which you humanely send to—for you *Austin Powers*

fans—an evil petting zoo. You then mate and breed the remaining rats, a process known as *crossover* in genetic programming.

The offspring of the remaining rats will be essentially the same size as their parents, so you need to mutate a few. While mutation is rare and usually results in a neutral-to-nonbeneficial trait (low weight, in this case), sometimes you'll successfully produce a bigger rat.

The whole process then becomes a big repeating loop, whether done organically or programmatically, making me wonder whether we really *are* just virtual beings in an alien simulation. At any rate, the end of the loop—the stop condition—is when the rats reach the desired size or you just can't stand dealing with rats anymore.

For input to your simulation, you'll need some statistics. Use the metric system since you're a scientist, mad or not. You already know that the average weight of a female bullmastiff is 50,000 grams, and you can find useful rat statistics in **Table 7-1**.

Table 7-1: Brown Rat Weight and Breeding Statistics

Parameter	Published values
Minimum weight	200 grams
Average weight (female)	250 grams
Average weight (male)	300–350 grams
Maximum weight	600 grams*
Number of pups per litter	8–12
Litters per year	4–13
Life span (wild, captivity)	1–3 years, 4–6 years

Parameter	Published values

*Exceptional individuals may reach 1,000 grams in captivity.

Because both domestic and wild brown rats exist, there may be wide variation in some of the stats. Rats in captivity tend to be better cared for than wild rats, so they weigh more, breed more, and have more pups. So you can choose from the higher end when a range is available. For this project, start with the assumptions in **Table 7-2**.

Table 7-2: Input Assumptions for the Super-Rats Genetic Algorithm

Variable and value	Comments
`GOAL = 50000`	Target weight in grams (female bullmastiff)
`NUM_RATS = 20`	Total number of *adult* rats your lab can support
`INITIAL_MIN_WT = 200`	Minimum weight of adult rat, in grams, in initial population
`INITIAL_MAX_WT = 600`	Maximum weight of adult rat, in grams, in initial population
`INITIAL_MODE_WT = 300`	Most common adult rat weight, in grams, in initial population
`MUTATE_ODDS = 0.01`	Probability of a mutation occurring in a rat
`MUTATE_MIN = 0.5`	Scalar on rat weight of least beneficial mutation
`MUTATE_MAX = 1.2`	Scalar on rat weight of most beneficial mutation
`LITTER_SIZE = 8`	Number of pups per pair of mating rats

Variable and value	Comments
`LITTERS_PER_YEAR = 10`	Number of litters per year per pair of mating rats
`GENERATION_LIMIT = 500`	Generational cutoff to stop breeding program

Since rats breed so frequently, you shouldn't have to factor in life span. Even though you will retain some of the parents from a previous generation, they will be culled out quickly as their offspring increase in weight from generation to generation.

The Super-Rats Code

The *super_rats.py* code follows the general workflow in **Figure 7-2**. You can also download the code from **https://www.nostarch.com/impracticalpython/**.

Entering the Data and Assumptions

Listing 7-1, in the global space at the start of the program, imports modules and assigns the statistics, scalars, and assumptions in **Table 7-2** as constants. Once the program is complete and working, feel free to experiment with the values in that table and see how they affect your results.

super_rats.py, part 1

```
❶ import time
   import random
   import statistics

❷ # CONSTANTS (weights in grams)
❸ GOAL = 50000
   NUM_RATS = 20
```

```
    INITIAL_MIN_WT = 200
    INITIAL_MAX_WT = 600
    INITIAL_MODE_WT = 300
    MUTATE_ODDS = 0.01
    MUTATE_MIN = 0.5
    MUTATE_MAX = 1.2
    LITTER_SIZE = 8
    LITTERS_PER_YEAR = 10
    GENERATION_LIMIT = 500

    # ensure even-number of rats for breeding pairs:
❹ if NUM_RATS % 2 != 0:
        NUM_RATS += 1
```

Listing 7-1: Imports modules and assigns constants

Start by importing the `time`, `random`, and `statistics` modules ❶. You'll use the `time` module to record the runtime of your genetic algorithm. It's interesting to time genetic algorithms, if only to be awed by how quickly they can find a solution.

The `random` module will satisfy the stochastic needs of the algorithm, and you'll use the `statistics` module to get mean values. This is a weak use for `statistics`, but I want you to be aware of the module, since it can be quite handy.

Next, assign the input variables described in **Table 7-2** and be sure to note that the units are grams ❷. Use uppercase letters for the names, as these represent constants ❸.

Right now, we're going to assume the use of breeding *pairs*, so check that the user input an even number of rats and, if not, add a rat ❹. Later, in "**Challenge Projects**" on **page 144**, you'll get to experiment with alternative gender distributions.

Initializing the Population

Listing 7-2 is the program's shopping representative. It goes to a pet shop and picks out the rats for an initial breeding population. Since you want mating pairs, it should choose an even number of rats. And since you can't afford one of those fancy volcano lairs with unlimited space, you'll need to maintain a constant number of adult rats through each generation—though the number can swell temporarily to accommodate litters. Remember, the rats will need more and more space as they grow to the size of big dogs!

super_rats.py, part 2

```
❶ def populate(num_rats, min_wt, max_wt, mode_wt):
      """Initialize a population with a triangular distribution of weights."""
   ❷ return [int(random.triangular(min_wt, max_wt, mode_wt))\
             for i in range(num_rats)]
```

Listing 7-2: Defines the function that creates the initial rat population

The `populate()` function needs to know the amount of adult rats you want, the minimum and maximum weights for the rats, and the most commonly occurring weight ❶. Note that all of these arguments will use constants found in the global space. You don't have to pass these as arguments for the function to access them. But I do so here and in the functions that follow, for clarity and because local variables are accessed more efficiently.

You'll use the four arguments above with the `random` module, which includes different types of distributions. You'll use a triangular distribution here, because it gives you firm control of the minimum and maximum sizes and lets you model skewness in the statistics.

Because brown rats exist both in the wild and in captivity—in zoos, labs, and as pets—their weights are skewed to the high side. Wild rats tend to be smaller as their lives are nasty, brutish, and short, though lab rats may contest that point! Use list comprehension to loop through the number of

rats and assign each one a weight. Bundle it all together with the `return` statement ❷.

Measuring the Fitness of the Population

Measuring the fitness of the rats is a two-step process. First, grade the whole population by comparing the average weight of all the rats to the bullmastiff target. Then, grade each individual rat. Only rats whose weight ranks in the upper *n* percent, as determined by the `NUM_RATS` variable, get to breed again. Although the average weight of the population is a valid fitness measurement, its primary role here is to determine whether it's time to stop looping and declare success.

Listing 7-3 defines the `fitness()` and `select()` functions, which together form the measurement portion of your genetic algorithm.

super_rats.py, part 3

```
❶ def fitness(population, goal):
      """Measure population fitness based on an attribute mean vs target."""
      ave = statistics.mean(population)
      return ave / goal

❷ def select(population, to_retain):
      """Cull a population to retain only a specified number of members."""
   ❸ sorted_population = sorted(population)
   ❹ to_retain_by_sex = to_retain//2
   ❺ members_per_sex = len(sorted_population)//2
   ❻ females = sorted_population[:members_per_sex]
      males = sorted_population[members_per_sex:]
   ❼ selected_females = females[-to_retain_by_sex:]
      selected_males = males[-to_retain_by_sex:]
   ❽ return selected_males, selected_females
```

Listing 7-3: Defines the measurement step of the genetic algorithm

Define a function to grade the fitness of the current generation ❶. Use the `statistics` module to get the mean of the population and return this value divided by the target weight. When this value is equal to or greater than 1, you'll know it's time to stop breeding.

Next, define a function that culls a population of rats, based on weight, down to the `NUM_RATS` value, represented here by the `to_retain` parameter ❷. It will also take a `population` argument, which will be the parents of each generation.

Now, sort the population so you can distinguish large from small ❸. Take the number of rats you want to retain and divide it by 2 using floor division so that the result is an integer ❹. Do this step so you can keep the biggest male and female rats. If you choose only the largest rats in the population, you will theoretically be choosing only males. You obtain the total members of the current population, by sex, by dividing the `sorted_population` by 2, again using floor division ❺.

Male rats tend to be larger than females, so make two simplifying assumptions: first, assume that exactly half of the population is female and, second, that the largest female rat is no heavier than the smallest male rat. This means that the first half of the sorted population list represents females and the last half represents males. Then create two new lists by splitting `sorted_population` in half, taking the bottom half for females ❻ and the upper half for males. Now all that's left to do is take the biggest rats from the end of each of these lists ❼—using negative slicing—and return them ❽. These two lists contain the parents of the next generation.

The first time you run this function, all it will do is sort the rats by sex, as the initial number of rats already equals the `NUM_RATS` constant. After that, the incoming population argument will include both parents and children, and its value will exceed `NUM_RATS`.

Breeding a New Generation

Listing 7-4 defines the program's "crossover" step, which means it breeds the next generation. A key assumption is that the weight of every child

will be greater than or equal to the weight of the mother and less than or equal to the weight of the father. Exceptions to that rule will be handled in the "mutation" function.

super_rats.py, part 4

```
❶ def breed(males, females, litter_size):
      """Crossover genes among members (weights) of a population."""
   ❷ random.shuffle(males)
      random.shuffle(females)
   ❸ children = []
   ❹ for male, female in zip(males, females):
       ❺ for child in range(litter_size):
           ❻ child = random.randint(female, male)
           ❼ children.append(child)
   ❽ return children
```

Listing 7-4: Defines the function that breeds a new generation of rats

The `breed()` function takes as arguments the lists of weights of selected males and females returned from the `select()` function along with the size of a litter ❶. Next, randomly shuffle the two lists ❷, because you sorted them in the `select()` function and iterating over them without shuffling would result in the smallest male being paired with the smallest female, and so on. You need to allow for love and romance; the largest male may be drawn to the most petite female!

Start an empty list to hold their children ❸. Now for the hanky-panky. Go through the shuffled lists using `zip()` to pair a male and female from each list ❹. Each pair of rats can have multiple children, so start another loop that uses the litter size as a range ❺. The litter size is a constant, called `LITTER_SIZE`, that you provided in the input parameters, so if the value is `8`, you'll get eight children.

For each child, choose a weight at random between the mother's and father's weights ❻. Note that you don't need to use `male` + 1, because

`randint()` uses *all* the numbers in the supplied range. Note also that the two values can be the same, but the first value (the mother's weight) can never be larger than the second (the father's weight). This is another reason for the simplifying assumption that females must be no larger than the smallest male. End the loop by appending each child to the list of children ❼, then return `children` ❽.

Mutating the Population

A small percentage of the children should experience mutations, and most of these should result in traits that are nonbeneficial. That means lower-than-expected weights, including "runts" that would not survive. But every so often, a beneficial mutation will result in a heavier rat.

Listing 7-5 defines the `mutate()` function, which applies the mutation assumptions you supplied in the list of constants. After `mutate()` is called, it will be time to check the fitness of the new population and start the loop over if the target weight hasn't been reached.

super_rats.py, part 5

```
❶ def mutate(children, mutate_odds, mutate_min, mutate_max):
      """Randomly alter rat weights using input odds & fractional changes."""
   ❷ for index, rat in enumerate(children):
         if mutate_odds >= random.random():
          ❸ children[index] = round(rat * random.uniform(mutate_min,
                                                         mutate_max))
      return children
```

Listing 7-5: Defines the function that mutates a small portion of the population

The function needs the list of children, the odds of a mutation occurring, and the minimum and maximum impacts of a mutation ❶. The impacts are scalars that you'll apply to the weight of a rat. In your list of constants at the start of the program (and in **Table 7-2**), they are skewed to the minimum side, as most mutations do not result in beneficial traits.

Loop through the list of children and use `enumerate()`—a handy built-in function that acts as an automatic counter—to get an index ❷. Then use the `random()` method to generate a random number between 0 and 1 and compare it to the odds of a mutation occurring.

If the `mutate_odds` variable is greater than or equal to the randomly generated number, the rat (weight) at that index is mutated. Choose a mutation value from a uniform distribution defined by the minimum and maximum mutation values; this basically selects a value at random from the min-max range. As these values are skewed to the minimum, the outcome is more likely to be a loss in weight than a gain. Multiply the current weight by this mutation scalar and round it to an integer ❸. Finish by returning the mutated `children` list.

NOTE

With regard to the validity of mutation statistics, you can find studies that suggest beneficial mutations are very rare and others that suggest they are more common than we realize. The breeding of dogs has shown that achieving drastic variations in size (for example, Chihuahuas vs. Great Danes) doesn't require millions of years of evolution. In a famous 20th-century study, Russian geneticist Dmitry Belyayev started with 130 silver foxes and, over a 40-year period, succeeded in achieving dramatic physiological changes by simply selecting the tamest foxes in each generation.

Defining the main() Function

Listing 7-6 defines the `main()` function, which manages the other functions and determines when you've met the stop condition. It will also display all the important results.

super_rats.py, part 6

```
def main():
    """Initialize population, select, breed, and mutate, display results."""
```

```
❶ generations = 0

❷ parents = populate(NUM_RATS, INITIAL_MIN_WT, INITIAL_MAX_WT,
                     INITIAL_MODE_WT)
   print("initial population weights = {}".format(parents))
   popl_fitness = fitness(parents, GOAL)
   print("initial population fitness = {}".format(popl_fitness))
   print("number to retain = {}".format(NUM_RATS))

❸ ave_wt = []

❹ while popl_fitness < 1 and generations < GENERATION_LIMIT:
       selected_males, selected_females = select(parents, NUM_RATS)
       children = breed(selected_males, selected_females, LITTER_SIZE)
       children = mutate(children, MUTATE_ODDS, MUTATE_MIN, MUTATE_MAX)
    ❺ parents = selected_males + selected_females + children
       popl_fitness = fitness(parents, GOAL)
    ❻ print("Generation {} fitness = {:.4f}".format(generations,
                                                    popl_fitness))
    ❼ ave_wt.append(int(statistics.mean(parents)))
       generations += 1
❽ print("average weight per generation = {}".format(ave_wt))
   print("\nnumber of generations = {}".format(generations))
   print("number of years = {}".format(int(generations / LITTERS_PER_YEAR)))
```

Listing 7-6: Defines the `main()` function

Start the function by initializing an empty list to hold the number of generations. You'll eventually use this to figure out how many years it took to achieve your goal ❶.

Next, call the `populate()` function ❷ and immediately print the results. Then, get the fitness of your initial population and print this along with the number of rats to retain each generation, which is the `NUM_RATS` constant.

For fun, initialize a list to hold the average weight of each generation so you can view it at the end ❸. If you plot these weights against the number of years, you'll see that the trend is exponential.

Now, start the big genetic loop of select-mate-mutate. This is in the form of a `while` loop, with the stop conditions being either reaching the target weight or reaching a large number of generations without achieving the target weight ❹. Note that after mutating the children, you need to combine them with their parents to make a new `parents` list ❺. It takes the pups about five weeks to mature and start breeding, but you can account for this by adjusting the `LITTERS_PER_YEAR` constant down from the maximum possible value (see **Table 7-1**), as we've done here.

At the end of each loop, display the results of the `fitness()` function to four decimal places so you can monitor the algorithm and ensure it is progressing as expected ❻. Get the average weight of the generation, append it to the `ave_wt` list ❼, and then advance the generation count by 1.

Complete the `main()` function by displaying the list of average weights per generation, the number of generations, and the number of years—calculated using the `LITTERS_PER_YEAR` variable ❽.

Running the main() Function

Finish up with the familiar conditional statement for running the program either stand-alone or as a module. Get the ending time and print how long it took the program to run. The performance information should print only when the module is run in stand-alone mode, so be sure to place it under the `if` clause. See **Listing 7-7**.

super_rats.py, part 7

```
if __name__ == '__main__':
    start_time = time.time()
    main()
    end_time = time.time()
```

```
    duration = end_time - start_time
    print("\nRuntime for this program was {} seconds.".format(duration))
```

Listing 7-7: Runs the `main()` function and `time` module if the program isn't imported

Summary

With the parameters in **Table 7-2**, the *super_rats.py* program will take about two seconds to run. On average, it will take the rats about 345 generations, or 34.5 years, to reach the target weight of 110 pounds. That's a long time for a mad scientist to stay mad! But armed with your program, you can look for ways to reduce the time to target.

Sensitivity studies work by making multiple changes to a *single* variable and judging the results. You should take care in the event some variables are dependent on one another. And since the results are stochastic (random), you should make multiple runs with each parameter change in order to capture the range of possible outcomes.

Two things you can control in your breeding program are the number of breeding rats (`NUM_RATS`) and the odds of a mutation occurring (`MUTATE_ODDS`). The mutation odds are influenced by factors like diet and exposure to radiation. If you change these variables one at a time and rerun *super_rats.py*, you can judge the impact of each variable on the project timeline.

An immediate observation is that, if you start with small values for each variable and slowly increase them, you get dramatic initial results (see **Figure 7-3**). After that, both curves decline rapidly and flatten out in a classic example of diminishing returns. The point where each curve flattens is the key to optimally saving money and reducing work.

For example, you get very little benefit from retaining more than about 300 rats. You'd just be feeding and caring for a lot of superfluous rats. Likewise, trying to boost the odds of a mutation above 0.3 gains you little.

With charts like these, it's easy to plan a path forward. The horizontal dotted line marked "Baseline" represents the average result of using the input in **Table 7-2**. You can potentially reduce this time by over 10 years just by retaining 50 rats rather than 20. You should also focus on increasing the number of beneficial mutations. This will be more rewarding, but riskier and harder to control.

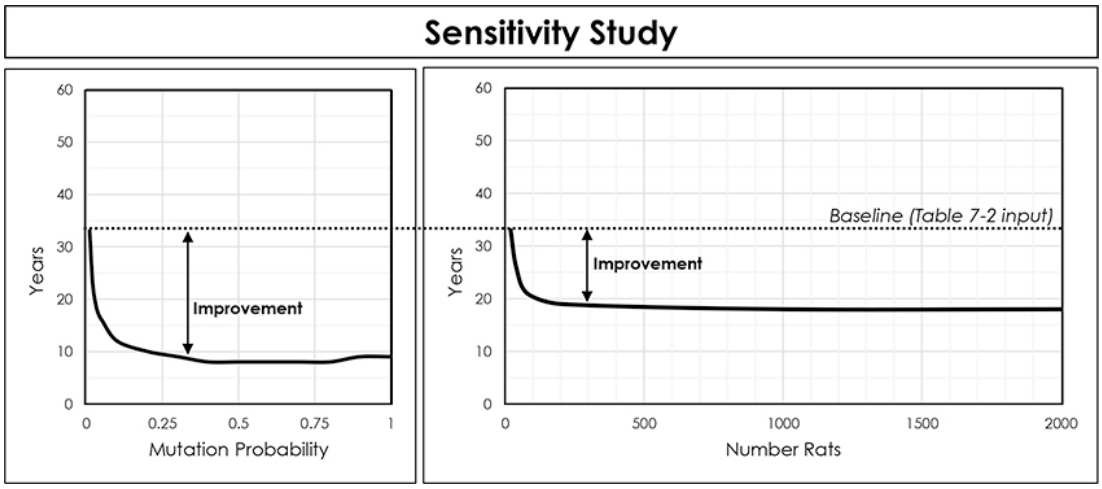

Figure 7-3: Impact of two parameters on the time required to reach the target weight

If you rerun the simulation using 50 rats and bumping the odds of mutation up to 0.05, you can theoretically complete the project in 14 years, an improvement of 246 percent over the initial baseline. Now *that's* optimization!

Breeding super-rats was a fun and simple way to understand the basics of genetic algorithms. But to truly appreciate their power, you need to attempt something harder. You need a brute-force problem that's too big to brute-force, and the next project is that kind of problem.

Project #14: Cracking a High-Tech Safe

You are Q, and James Bond has a problem. He has to attend an elegant dinner party at a villain's mansion, slip away to the man's private office, and crack his wall safe. Child's play for 007, except for one thing: it's a Humperdink BR549 digital safe that takes 10 digits, yielding 10 billion possible combinations. And the lock wheels don't start turning until *after*

all the numbers have been entered. There'll be no putting a stethoscope to this safe and slowly turning a dial!

As Q, you already have an autodialer device that can brute-force its way through all possible solutions, but Bond simply won't have time to use it. Here's why.

A combination lock should really be called a *permutation* lock, because it requires *ordered* combinations, which are, by definition, permutations. More specifically, locks rely on *permutations with repetition*. For example, a valid—though insecure—combination could be 999999999.

You used the `itertools` module's `permutations()` iterator when working with anagrams in **Chapter 3** and in "**Practice Projects**" on **page 87** in **Chapter 4**, but that won't help here because `permutations()` returns permutations *without* repetition. To generate the right kind of permutation for a lock, you need to use `itertools`'s `product()` iterator, which calculates the Cartesian product from multiple sets of numbers:

```
>>> from itertools import product
>>> combo = (1, 2)
>>> for perm in product(combo, repeat=2):
    print(perm)
(1, 1)
(1, 2)
(2, 1)
(2, 2)
```

The optional `repeat` keyword argument lets you take the product of an iterable multiplied by itself, as you need to do in this case. Note that the `product()` function returns all the possible combinations, whereas the `permutations()` function would return only `(1, 2)` and `(2, 1)`. You can read more about `product()` at **https://docs.python.org/3.6/library/itertools.html#itertools.product.Listing**.

Listing 7-8 is a Python program, called *brute_force_cracker.py*, that uses `product()` to brute-force its way to the right combination:

brute_force_cracker.py

```
❶ import time
   from itertools import product

   start_time = time.time()

❷ combo = (9, 9, 7, 6, 5, 4, 3)

   # use Cartesian product to generate permutations with repetition
❸ for perm in product([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], repeat=len(combo)):
    ❹ if perm == combo:
            print("Cracked! {} {}".format(combo, perm))

   end_time = time.time()
❺ print("\nRuntime for this program was {} seconds.".format
         (end_time - start_time))
```

Listing 7-8: Uses a brute-force method to find a safe's combination

Import `time` and the `product` iterator from `itertools` ❶. Get the start time, and then enter the safe combination as a tuple ❷. Next use `product()`, which returns tuples of all the permutations with repetition for a given sequence. The sequence contains all the valid single-digit entries (0–9). You should set the `repeat` argument to the number of digits in the combination ❸. Compare each result to the combination and print `"Cracked!"` if they match, along with the combination and matching permutation ❹. Finish by displaying the runtime ❺.

This works great for combinations up to eight digits long. After that, the wait becomes increasingly uncomfortable. **Table 7-3** is a record of runtimes for the program versus number of digits in the combination.

Table 7-3: Runtimes Versus Digits in Combination (2.3 GHz Processor)

Number of digits	Runtime in seconds
5	0.035
6	0.147
7	1.335
8	12.811
9	133.270
10	1396.955

Notice that adding a number to the combination increases the runtime by an order of magnitude. This is an exponential increase. With 9 digits, you'd wait over 2 minutes for an answer. With 10 digits, over 20 minutes! That's a long time for Bond to take an unnoticed "bathroom break."

Fortunately, you're Q, and you know about genetic algorithms. All you need is some way to judge the fitness of each candidate combination. Options include monitoring fluctuations in power consumption, measuring time delays in operations, and listening for sounds. Let's assume use of a sound-amplifying tool, along with a tool to prevent lockouts after a few incorrect combinations have been entered. Because of the safeguards in the BR549 safe, a sound tool can initially tell you only *how many* digits are correct, not *which* digits, but with very little time, your algorithm can zero in on the solution.

THE OBJECTIVE

Use a genetic algorithm to quickly find a safe's combination in a large search space.

Strategy

The strategy here is straightforward. You'll generate a sequence of 10 numbers at random and compare it to the real combination, grading the result based on matches; in the real world, you'd find the number of matches using the sound detector clamped to the door of the safe. You then change one value in your solution and compare again. If another match is found, you throw away the old sequence and move forward with the new; otherwise, you keep the old sequence and try again.

Since one solution completely replaces the other, this represents 100 percent crossover of genetic material, so you are essentially using just selection and mutation. Selection plus mutation alone generates a robust *hill-climbing* algorithm. Hill climbing is an optimization technique that starts with an arbitrary solution and changes (mutates) a single value in the solution. If the result is an improvement, the new solution is kept and the process repeats.

A problem with hill climbing is that the algorithm can get stuck in *local* minima or maxima and not find the optimal *global* value. Imagine you are looking for the lowest value in the wavelike function in **Figure 7-4**. The current best guess is marked by the large black dot. If the magnitude of the change you are making (mutation) is too small to "escape" the local trough, the algorithm won't find the true low point. From the algorithm's point of view, because every direction results in a worse answer, it must have found the true answer. So it prematurely converges on a solution.

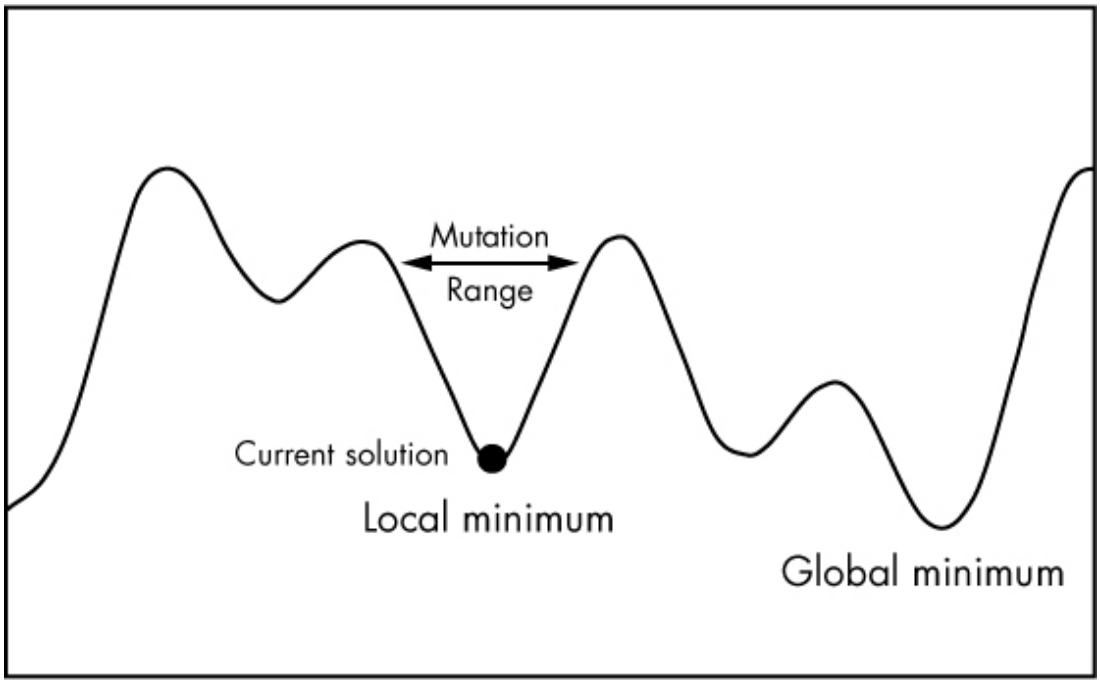

Figure 7-4: Example of a hill-climbing algorithm "stuck" in a local minimum

Using crossover in genetic algorithms helps to avoid premature convergence problems, as does allowing for relatively large mutations. Because you're not worried about honoring biological realism here, the mutation space can encompass every possible value in the combination. That way you can't get stuck, and hill climbing is an acceptable approach.

The Safecracker Code

The *safe_cracker.py* code takes a combination of *n* digits and uses hill climbing to reach the combination from a random starting point. The code can be downloaded from **https://www.nostarch.com/impracticalpython/**.

Setting Up and Defining the fitness() Function

Listing 7-9 imports the necessary modules and defines the `fitness()` function.

safe_cracker.py, part 1

```
❶ import time
   from random import randint, randrange
```

```
❷ def fitness(combo, attempt):
      """Compare items in two lists and count number of matches."""
      grade = 0
   ❸ for i, j in zip(combo, attempt):
          if i == j:
              grade += 1
      return grade
```

Listing 7-9: Imports modules and defines the `fitness()` function

After importing some familiar modules ❶, define a `fitness()` function that takes the true combination and an attempted solution as arguments ❷. Name a variable `grade` and set it to `0`. Then use `zip()` to iterate through each element in the combination and your attempt ❸. If they're the same, add 1 to `grade` and return it. Note that you aren't recording the index that matches, just that the function has found a match. This emulates output from the sound detection device. All it can tell you initially is how many lock wheels turned, not their locations.

Defining and Running the main() Function

Since this is a short and simple program, most of the algorithm is run in the `main()` function, **Listing 7-10**, rather than in multiple functions.

safe_cracker.py, part 2

```
def main():
    """Use hill-climbing algorithm to solve lock combination."""
 ❶ combination = '6822858902'
    print("Combination = {}".format(combination))
    # convert combination to list:
 ❷ combo = [int(i) for i in combination]

    # generate guess & grade fitness:
 ❸ best_attempt = [0] * len(combo)
    best_attempt_grade = fitness(combo, best_attempt)
```

```
    ❹ count = 0

        # evolve guess
    ❺ while best_attempt != combo:
            # crossover
        ❻ next_try = best_attempt[:]

            # mutate
            lock_wheel = randrange(0, len(combo))
        ❼ next_try[lock_wheel] = randint(0, 9)

            # grade & select
        ❽ next_try_grade = fitness(combo, next_try)
            if next_try_grade > best_attempt_grade:
                best_attempt = next_try[:]
                best_attempt_grade = next_try_grade
            print(next_try, best_attempt)
            count += 1

        print()
    ❾ print("Cracked! {}".format(best_attempt), end=' ')
        print("in {} tries!".format(count))
    if __name__ == '__main__':
        start_time = time.time()
        main()
        end_time = time.time()
        duration = end_time - start_time
    ❿ print("\nRuntime for this program was {:.5f} seconds.".format(duration))
```

Listing 7-10: Defines the `main()` function and runs and times the program if it hasn't been imported

Provide the true combination as a variable ❶ and use list comprehension to convert it into a list for convenience going forward ❷. Generate a list of zeros equal in length to the combination and name it `best_attempt` ❸. At this point, any combination is as good as any other. You should retain this

name—`best_attempt`—because you need to preserve only the best solution as you climb the hill. Once you've generated the initial attempt, grade it with the `fitness()` function and then assign the value to a variable, called `best_attempt_grade`.

Start a `count` variable at zero. The program will use this variable to record how many attempts it took to crack the code ❹.

Now, start a `while` loop that continues until you've found the combination ❺. Assign a *copy* of `best_attempt` to a `next_try` variable ❻. You copy it so you don't run into aliasing problems; when you alter an element in `next_try`, you don't want to accidentally change `best_attempt`, because you may continue to use it in the event `next_try` fails the fitness test.

It's now time to mutate the copy. Each digit in the combination turns a lock wheel in the safe, so name a variable `lock_wheel` and randomly set it equal to an index location in the combination. This represents the location of the single element to change in this iteration. Next, randomly choose a digit and use it to replace the value at the location indexed by `lock_wheel` ❼.

Grade `next_try`, and if it's fitter than the previous attempt, reset both `best_attempt` and `best_attempt_grade` to the new values ❽. Otherwise, `best_attempt` will remain unchanged for the next iteration. Print both `next_try` and `best_attempt`, side by side, so you can scroll through the attempts when the program ends and see how they evolved. Finish the loop by advancing the counter.

When the program finds the combination, display the `best_attempt` value and the number of tries it took to find it ❾. Remember, the `end=' '` argument prevents a carriage return at the end of the printed line and places a space between the end of the current line and the beginning of the next line.

Complete the program with the conditional statement for running `main()` stand-alone and display the runtime to five decimal places ❿. Note that

the timing code comes after the conditional, and thus will not run if the program is imported as a module.

Summary

The last few lines of output from the *safe_cracker.py* program are shown here. I've omitted most of the evolving comparisons for brevity. The run was for a 10-digit combination.

```
[6, 8, 6, 2, 0, 5, 8, 9, 0, 0] [6, 8, 2, 2, 0, 5, 8, 9, 0, 0]
[6, 8, 2, 2, 0, 9, 8, 9, 0, 0] [6, 8, 2, 2, 0, 5, 8, 9, 0, 0]
[6, 8, 2, 2, 8, 5, 8, 9, 0, 0] [6, 8, 2, 2, 8, 5, 8, 9, 0, 0]
[6, 8, 2, 2, 8, 5, 8, 9, 0, 2] [6, 8, 2, 2, 8, 5, 8, 9, 0, 2]

Cracked! [6, 8, 2, 2, 8, 5, 8, 9, 0, 2] in 78 tries!

Runtime for this program was 0.69172 seconds.
```

Ten billion possible combinations, and the program found a solution in only 78 tries and in less than a second. Even James Bond would be impressed with that.

That does it for genetic algorithms. You've used an example workflow to breed gigantic rodents, then trimmed it to hill climb through a brute-force problem in no time flat. If you want to continue to play digital Darwin and experiment with genetic algorithms, a long list of example applications can be found on Wikipedia (**https://en.wikipedia.org/wiki/List_of_genetic_algorithm_applications**). Examples include:

- Modeling global temperature changes
- Container-loading optimization
- Delivery vehicle–routing optimization
- Groundwater-monitoring networks
- Learning robot behavior
- Protein folding
- Rare-event analysis

- Code breaking
- Clustering for fit functions
- Filtering and signal processing

Further Reading

Genetic Algorithms with Python (Amazon Digital Services LLC, 2016) by Clinton Sheppard is a beginner-level introduction to genetic algorithms using Python. It is available in paperback or as an inexpensive ebook from **https://leanpub.com/genetic_algorithms_with_python/**.

Challenge Projects

Continue to breed super-rats and crack super-safes with these suggested projects. As usual with challenge projects, you're on your own; no solutions are provided.

Building a Rat Harem

Since a single male rat can mate with multiple females, there is no need to have an equal number of male and female rats. Rewrite the *super_rats.py* code to accommodate a variable number of male and female individuals. Then rerun the program with the same total number of rats as before, but use 4 males and 16 females. How does this impact the number of years required to reach the target weight of 50,000 grams?

Creating a More Efficient Safecracker

As currently written, when a lock wheel match is found by the *safe_cracker.py* code, that match is not explicitly preserved. As long as the `while` loop is running, there's nothing to stop a correct match from being stochastically overwritten. Alter the code so that the indexes for correct guesses are excluded from future changes. Do timing comparisons between the two versions of the code to judge the impact of the change.